

NT-C-015 | Core Java | Course Knowledge Profile

Identity, purpose, prerequisites, scope and boundaries

Category	Programming
Target learners	Beginners and learners preparing for Java application development.
Prerequisites	Basic computer use and logical thinking.
Approved duration	10-12 weeks / 100-140 guided hours.
Final outcome	Create tested Java applications using OOP, collections, files and database access.
Assessment profile	Programming/Platform

Approved tools / platforms

Java, JDK, collections, streams, JDBC, SQL, JUnit, Maven or Gradle, Git.

Knowledge boundaries

Include	Quality rule	Exclude / escalate
The eight approved modules, four sections, named tools, projects and mapped entry-level roles.	Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.	Do not treat copied code or syntax recall as proof of implementation skill.
Current product/platform procedures only when version and date are known.	Use primary documentation and record the source version.	Do not invent current commands, prices, tax rates, policies or certification status.
Evidence-based career guidance.	Cite tests, projects, capstone, viva and verified resume evidence.	No guaranteed job, placement, salary, interview or employer claim.

LLM answer pattern: identify the course and version, answer from approved records, cite record IDs, distinguish verified facts from guidance, and state any missing evidence.

NT-C-015 | Core Java | NT-C-015-S01 Deep Knowledge

Modules 1-2 | objectives, concepts, evidence and remediation

Section competency	JDK, syntax, types and operators; Conditions, loops, arrays and methods
Completion evidence	Basic programs; Logic exercises
Section weight	20% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 1 - Java setup

Objective: JDK, syntax, types and operators

Observable evidence: Basic programs

Concept	Approved knowledge statement	Application behaviour	Evidence
JDK	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when jdk is appropriate; apply it in a Core Java task; verify the result.	Basic programs
syntax	The formal rules used to write valid instructions in a programming language. In Core Java, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when syntax is appropriate; apply it in a Core Java task; verify the result.	Basic programs
types	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when types is appropriate; apply it in a Core Java task; verify the result.	Basic programs
operators	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when operators is appropriate; apply it in a Core Java task; verify the result.	Basic programs

Module 2 - Control and methods

Objective: Conditions, loops, arrays and methods

Observable evidence: Logic exercises

Concept	Approved knowledge statement	Application behaviour	Evidence
Conditions	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when conditions is appropriate; apply it in a Core Java task; verify the result.	Logic exercises
loops	A design approach that organises behaviour and data around objects and clear responsibilities. In Core Java, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when loops is appropriate; apply it in a Core Java task; verify the result.	Logic exercises
arrays	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when arrays is appropriate; apply it in a Core Java task; verify the result.	Logic exercises
methods	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when methods is appropriate; apply it in a Core Java task; verify the result.	Logic exercises

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Basic programs; Logic exercises, return to Modules 1-2, complete one guided practice and then use a new equivalent test form.

NT-C-015 | Core Java | NT-C-015-S02 Deep Knowledge

Modules 3-4 | objectives, concepts, evidence and remediation

Section competency	Classes, inheritance, interfaces and polymorphism; Lists, sets, maps and type safety
Completion evidence	OOP application; Collection-based program
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 3 - Object-oriented Java

Objective: Classes, inheritance, interfaces and polymorphism

Observable evidence: OOP application

Concept	Approved knowledge statement	Application behaviour	Evidence
Classes	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when classes is appropriate; apply it in a Core Java task; verify the result.	OOP application
inheritance	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when inheritance is appropriate; apply it in a Core Java task; verify the result.	OOP application
interfaces	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when interfaces is appropriate; apply it in a Core Java task; verify the result.	OOP application
polymorphism	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when polymorphism is appropriate; apply it in a Core Java task; verify the result.	OOP application

Module 4 - Collections and generics

Objective: Lists, sets, maps and type safety

Observable evidence: Collection-based program

Concept	Approved knowledge statement	Application behaviour	Evidence
Lists	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when lists is appropriate; apply it in a Core Java task; verify the result.	Collection-based program
sets	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when sets is appropriate; apply it in a Core Java task; verify the result.	Collection-based program
maps	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when maps is appropriate; apply it in a Core Java task; verify the result.	Collection-based program
type safety	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when type safety is appropriate; apply it in a Core Java task; verify the result.	Collection-based program

Remediation: If the learner cannot explain the purpose, complete the stated task or provide OOP application; Collection-based program, return to Modules 3-4, complete one guided practice and then use a new equivalent test form.

NT-C-015 | Core Java | NT-C-015-S03 Deep Knowledge

Modules 5-6 | objectives, concepts, evidence and remediation

Section competency	Error handling, I/O and logging; Lambdas, streams and date/time
Completion evidence	Reliable file utility; Data-processing task
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 5 - Exceptions and files

Objective: Error handling, I/O and logging

Observable evidence: Reliable file utility

Concept	Approved knowledge statement	Application behaviour	Evidence
Error handling	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when error handling is appropriate; apply it in a Core Java task; verify the result.	Reliable file utility
I/O	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when i/o is appropriate; apply it in a Core Java task; verify the result.	Reliable file utility
logging	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when logging is appropriate; apply it in a Core Java task; verify the result.	Reliable file utility

Module 6 - Modern Java

Objective: Lambdas, streams and date/time

Observable evidence: Data-processing task

Concept	Approved knowledge statement	Application behaviour	Evidence
Lambdas	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when lambdas is appropriate; apply it in a Core Java task; verify the result.	Data-processing task
streams	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when streams is appropriate; apply it in a Core Java task; verify the result.	Data-processing task
date/time	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when date/time is appropriate; apply it in a Core Java task; verify the result.	Data-processing task

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Reliable file utility; Data-processing task, return to Modules 5-6, complete one guided practice and then use a new equivalent test form.

NT-C-015 | Core Java | NT-C-015-S04 Deep Knowledge

Modules 7-8 | objectives, concepts, evidence and remediation

Section competency	Database access, CRUD and transactions; JUnit, build tools, Git and documentation
Completion evidence	Database-backed app; Tested Java project
Section weight	30% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 7 - JDBC and SQL

Objective: Database access, CRUD and transactions

Observable evidence: Database-backed app

Concept	Approved knowledge statement	Application behaviour	Evidence
Database access	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when database access is appropriate; apply it in a Core Java task; verify the result.	Database-backed app
CRUD	The create, read, update and delete lifecycle for persistent records. In Core Java, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when crud is appropriate; apply it in a Core Java task; verify the result.	Database-backed app
transactions	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when transactions is appropriate; apply it in a Core Java task; verify the result.	Database-backed app

Module 8 - Testing and capstone

Objective: JUnit, build tools, Git and documentation

Observable evidence: Tested Java project

Concept	Approved knowledge statement	Application behaviour	Evidence
JUnit	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when junit is appropriate; apply it in a Core Java task; verify the result.	Tested Java project
build tools	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when build tools is appropriate; apply it in a Core Java task; verify the result.	Tested Java project
Git	Distributed version control for tracking and reviewing source changes. In Core Java, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when git is appropriate; apply it in a Core Java task; verify the result.	Tested Java project
documentation	A defined competency within Core Java. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when documentation is appropriate; apply it in a Core Java task; verify the result.	Tested Java project

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Database-backed app; Tested Java project, return to Modules 7-8, complete one guided practice and then use a new equivalent test form.

NT-C-015 | Core Java | Skill Ontology & Dependency Map

Atomic skills, learning order and evidence strength

Skill ID	Competency	Depends on	Evidence	Type
NT-C-015-SK01	Java setup	Prerequisite foundation	Basic programs	Critical
NT-C-015-SK02	Control and methods	M01 plus earlier foundations	Logic exercises	Critical
NT-C-015-SK03	Object-oriented Java	M02 plus earlier foundations	OOP application	Supporting
NT-C-015-SK04	Collections and generics	M03 plus earlier foundations	Collection-based program	Supporting
NT-C-015-SK05	Exceptions and files	M04 plus earlier foundations	Reliable file utility	Supporting
NT-C-015-SK06	Modern Java	M05 plus earlier foundations	Data-processing task	Critical
NT-C-015-SK07	JDBC and SQL	M06 plus earlier foundations	Database-backed app	Supporting
NT-C-015-SK08	Testing and capstone	M07 plus earlier foundations	Tested Java project	Critical

Evidence-strength hierarchy

Strength	Evidence source	Use
High	Trainer-observed practical, viva, working source/project files and reproducible output.	May support verified mastery when rubric threshold is met.
Medium	Scored mock tests, reviewed assignments and documented portfolio artifacts.	Supports mastery with corroboration.
Low	Resume claim, self-report, attendance or video completion without demonstration.	May trigger verification; cannot alone support Apply Now.

CareerTours must not treat attendance, time spent or content opened as skill mastery.

NT-C-015 | Core Java | Assessment Knowledge & Sample Items

Non-live examples for LLM training and evaluation

Sample ID	Type	Question	Approved answer / rubric anchor
NT-C-015-S01-EX01	Evidence/application	What evidence best demonstrates the combined competency of Java setup and Control and methods?	The learner should provide Basic programs; Logic exercises and explain how the work applies JDK, syntax, types and operators; Conditions, loops, arrays and methods. Recall alone is insufficient.
NT-C-015-S01-EX02	Scenario/remediation	A learner reports completing Control and methods but cannot produce Logic exercises. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-015-S01, request Logic exercises, and allow a new equivalent test only after remediation.
NT-C-015-S02-EX01	Evidence/application	What evidence best demonstrates the combined competency of Object-oriented Java and Collections and generics?	The learner should provide OOP application; Collection-based program and explain how the work applies Classes, inheritance, interfaces and polymorphism; Lists, sets, maps and type safety. Recall alone is insufficient.
NT-C-015-S02-EX02	Scenario/remediation	A learner reports completing Collections and generics but cannot produce Collection-based program. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-015-S02, request Collection-based program, and allow a new equivalent test only after remediation.
NT-C-015-S03-EX01	Evidence/application	What evidence best demonstrates the combined competency of Exceptions and files and Modern Java?	The learner should provide Reliable file utility; Data-processing task and explain how the work applies Error handling, I/O and logging; Lambdas, streams and date/time. Recall alone is insufficient.
NT-C-015-S03-EX02	Scenario/remediation	A learner reports completing Modern Java but cannot produce Data-processing task. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-015-S03, request Data-processing task, and allow a new equivalent test only after remediation.
NT-C-015-S04-EX01	Evidence/application	What evidence best demonstrates the combined competency of JDBC and SQL and Testing and capstone?	The learner should provide Database-backed app; Tested Java project and explain how the work applies Database access, CRUD and transactions; JUnit, build tools, Git and documentation. Recall alone is insufficient.
NT-C-015-S04-EX02	Scenario/remediation	A learner reports completing Testing and capstone but cannot produce Tested Java project. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-015-S04, request Tested Java project, and allow a new equivalent test only after remediation.

Live bank rule: Generate at least three times the live quantity for each section using the approved Programming/Platform blueprint. Human-review every scored item before initial production release.

NT-C-015 | Core Java | Labs, Projects & Scoring Rubrics

Practical proof of learning and ownership

Level	Approved project	Skills evidenced	Required deliverables
Mini 1	Library manager	OOP and collections	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Mini 2	Inventory app	JDBC, SQL and transactions	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Capstone	Java business application	Testing, build and documentation	Brief; working/source file or reproducible environment; output; validation notes; reflection.

Standard project rubric

Dimension	Weight	What earns credit
Problem framing	15%	Clear objective, user/business need, assumptions and success criteria.
Technical/design accuracy	25%	Correct methods, appropriate tools and sound decisions.
Completeness	15%	Required features, assets, data or workflow are present.
Testing and validation	15%	Important normal, edge, quality or failure cases are checked.
Evidence and reproducibility	15%	Source/working files, setup, inputs, outputs and version information are available.
Documentation	5%	Concise structure, instructions and limitations.
Viva / ownership	10%	Learner can explain decisions, demonstrate work and respond to a change request.

Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.

NT-C-015 | Core Java | Career Roles, Evidence & Gap Actions

Explainable top-three occupation matching

Rank	Occupation	Default weighted skills	Minimum evidence	Gap action
1	Junior Java Programmer	Core Java 34% [critical]; OOP 33% [critical]; SQL 33%	Tested repository	If a critical skill is below 60 or tested repository is missing, return Improve First or Needs Evidence with the linked module/project.
2	Application Support Trainee - Java	Debugging 34% [critical]; files 33% [critical]; database 33%	Support-ready application	If a critical skill is below 60 or support-ready application is missing, return Improve First or Needs Evidence with the linked module/project.
3	Backend Developer Trainee	Java 34% [critical]; JDBC 33% [critical]; testing 33%	Database capstone	If a critical skill is below 60 or database capstone is missing, return Improve First or Needs Evidence with the linked module/project.

Course readiness	45% section mocks + 20% mini projects + 25% capstone + 10% viva.
Role readiness	45% role-relevant mastery + 25% projects/capstone + 20% aligned mocks + 10% verified resume evidence.
Evidence confidence	Show separately; Apply Now requires at least 70.
Resume extraction	Use only relevant skills, projects, education, experience and portfolio evidence; minimise personal data before external model processing.

A student may be Apply Now for one mapped occupation and Improve First for another. Return independent evidence and gaps for each role.

NT-C-015 | Core Java | FAQs, Misconceptions & LLM Response Pack

Course-specific retrieval and response patterns

Approved FAQ	Answer
Who is Core Java for?	Beginners and learners preparing for Java application development.
What should I know before starting?	Basic computer use and logical thinking.
How long is the approved learning journey?	10-12 weeks / 100-140 guided hours.
What will I be able to demonstrate?	Create tested Java applications using OOP, collections, files and database access.
Does completion guarantee a job?	No. Completion and role readiness are evidence-based guidance; employment, placement, salary and employer selection are not guaranteed.
What if I fail a section?	CareerTours identifies the failed skill tags and module, assigns targeted practice, and permits an equivalent retest after remediation.

Misconception	Approved correction
Attendance proves mastery	False - require tests and observable practical/project evidence.
One total score proves every role fit	False - calculate each occupation using its weighted skills and evidence.
The LLM can add missing details	False - retrieve approved records or state that evidence is insufficient.
Course completion guarantees employment	False - never claim guaranteed job, placement, salary or interview.
Core Java knowledge never changes	False - version-sensitive tools, laws, taxes, security and platform procedures require review.

Retrieval metadata

```
course_code=NT-C-015; corpus_version=3.0;
record_types=course_profile,module,section,skill,assessment_sample,project,occupation_mapping,faq,misconception;
approval_status=approved; effective_date=2026-08-12
```